

Lecture 20: RNN Foundations

Networks with memory - and why the memory fades

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

The reviews arrive

The [51] classifier grades each pomegranate; the boxes ship worldwide. Now the **reviews** arrive: "Fresh and sweet!" (3 words); an 11-word one naming Yerevan; a 42-word complaint; some in Armenian. Ops wants them read, translated, answered.

Zoom out: **every** headline AI system of the 2020s - ChatGPT included - is a **sequence model**. This chapter and the next are the road there.

Predict: feed a review into an MLP ([41_2]) - what breaks before you even train?

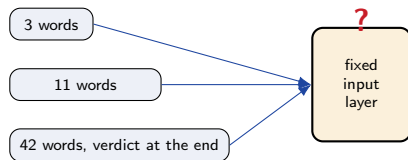
The reviews arrive

The [51] classifier grades each pomegranate; the boxes ship worldwide. Now the **reviews** arrive: "Fresh and sweet!" (3 words); an 11-word one naming Yerevan; a 42-word complaint; some in Armenian. Ops wants them read, translated, answered.

Zoom out: **every** headline AI system of the 2020s - ChatGPT included - is a **sequence model**. This chapter and the next are the road there.

Predict: feed a review into an MLP ([41_2]) - what breaks before you even train?

The input layer has a **fixed size**. Reviews do not.



Outline

Text refuses to fit

The recurrence idea

Training - and the fatal flaw

Text refuses to fit

Order is information, and length is a variable.

The sequence zoo

Text leads this chapter. Two close cousins get one line each: **time series** (tonnage logs, exchange rates) and **audio** (speech, music) share the same shape of problem.

What they share: **order matters**, **length varies**, and dependencies can be **long-range** - the word that resolves a sentence's meaning may be 40 tokens back.

Text

reviews, chats, articles

Time series

tonnage logs, FX rates

Audio

speech, music

The shuffle test

Take the review sentence and shuffle its words with one fixed permutation. Every word is identical; all meaning is gone.

The bag-of-words histograms are **pixel-identical**. Any order-blind model literally cannot tell the two apart.

Callback: [48] did this to an image's *pixels*. Same demo, new axis.

Same words, all meaning gone - identical histograms

original	The	pomegranates	arrived	fresh	and	sweet
shuffled	sweet	fresh	arrived	The	pomegranates	and
	was 6	was 4	was 3	was 1	was 2	was 5



The shoehorn hacks

How people force variable-length text into a fixed net - and the price of each:

Pad + truncate to a fixed length: waste on the short one, amputation on the long one

fixed length = 8

short
(3 words)

Fresh	and	sweet!	<pad>	<pad>	<pad>	<pad>	<pad>
-------	-----	--------	-------	-------	-------	-------	-------

medium
(11 words)

The	pomegranates	arrived	fresh	and	sweet,	best	I've
-----	--------------	---------	-------	-----	--------	------	------

LOST (3 words): had since Yerevan.

long
(42 words)

I	ordered	a	large	box	of	exported	pomegranates
---	---------	---	-------	-----	----	----------	--------------

LOST (34 words): for my mother's birthday expecting the same deep red arils and rich tart sweetness I remembered from childhood markets back home but after the long shipping delay they arrived bruised dry and honestly disappointing

(2) **Bag-of-words averaging** fails the shuffle test by construction - it cannot see order at all. (3) **A separate weight per position** means "I went to Munich in 2009" and "In 2009, I went to Munich" must each relearn the same fact at a different position.

What about a fixed window?

You have already met two models that read a sequence through a **fixed window** of the last k tokens:

- ▶ the **1D convolution** from [48] - slide the same kernel along a sentence instead of a signal, and it reads a window of k tokens;
- ▶ the ch11 **name inventor** - an MLP that predicts the next letter from the last k letters (Bengio et al., 2003).

Both learn **local** patterns well - "-yan" closes an Armenian surname. Both are **blind beyond** k :

The **pomegranates** that my grandmother sent us **from Goris last autumn** **were** sweet.

A 4-word window in front of "were" sees only *from Goris last autumn*. It cannot tell whether one pomegranate or many was sent - the word that decides it is 9 words back.

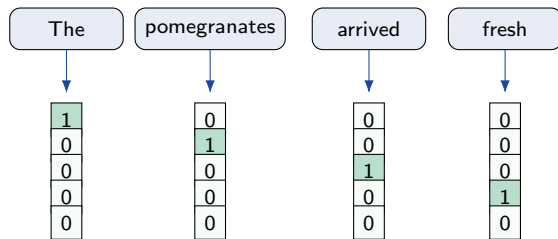
Stacking conv layers widens the window ([50]'s receptive field), but the width is fixed by the architecture before any text arrives. We want a memory that grows *with* the text: read one token at a time and carry a summary forward.

Tokens, minimally

To feed text to *any* network, words must first become vectors. For now, the simplest scheme: build a **vocabulary**, then represent each word as a **one-hot** vector.

Callback: [03] one-hot encoded categorical features. Same trick, new victim - a *word* instead of a category.

The full tokenization story (subwords, vocabulary explosion, the Armenian fact) comes in the LLM chapter.



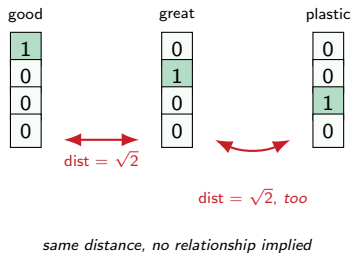
vocabulary size 5: one-hot columns

One-hot and its poverty

One-hot is all this chapter needs. It has one flaw worth seeing now:

In a 10,000-dimensional one-hot space, **every pair of words is equally far apart**. "good" is as far from "great" as it is from "plastic" - the geometry carries zero meaning.

Learned, dense word vectors fix this. They arrive with the attention chapter.



The recurrence idea

Read like a human: one word at a time, with memory.

You have already run a recurrence

[29]'s **exponential smoothing** forecast tomorrow from a running summary of the whole past:

$$\hat{y}_{t+1} = \alpha y_t + (1 - \alpha) \hat{y}_t$$

New summary = a little of today + most of the old summary. Unroll it and each older reading's weight shrinks by the same factor every step: with $\alpha = 0.2$ the weights are 0.20, 0.16, 0.13, 0.10, ... - each one 0.8 of the one before.

	smoothing, [29]	RNN, today
summary	one number $\hat{y}_t$	a vector $\mathbf{z}^{[t]}$
input	one reading y_t	a word's vector $\mathbf{x}^{[t]}$
what to keep	fixed: $1 - \alpha$	learned: a matrix $\mathbf{V}$
nonlinearity	none	tanh

An RNN is exponential smoothing that **learns what to keep**. Hold on to that 0.8: it comes back at the end of this lecture.

One new arrow

How a human reads: word by word, retaining a running summary - the **state**.

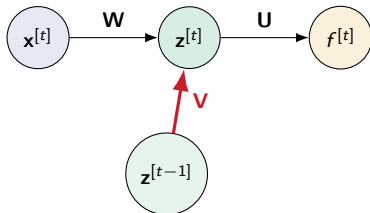
From [41_1]'s hidden layer $\mathbf{z} = \sigma(\mathbf{W}^\top \mathbf{x} + \mathbf{b})$ to

$$\mathbf{z}^{[t]} = \sigma(\mathbf{V}^\top \mathbf{z}^{[t-1]} + \mathbf{W}^\top \mathbf{x}^{[t]} + \mathbf{b}) :$$

one new term - the **hidden-to-hidden weights $\mathbf{V}$** - and the result is a **recurrent neural network (RNN)**.

$\mathbf{z}^{[t]}$ is a function of the current input *and* the previous state - by recurrence, a summary of **everything read so far**.

History: Elman (1990), "Finding Structure in Time", after Jordan (1986) - the idea predates the web.



Red: the one new arrow, hidden to hidden.

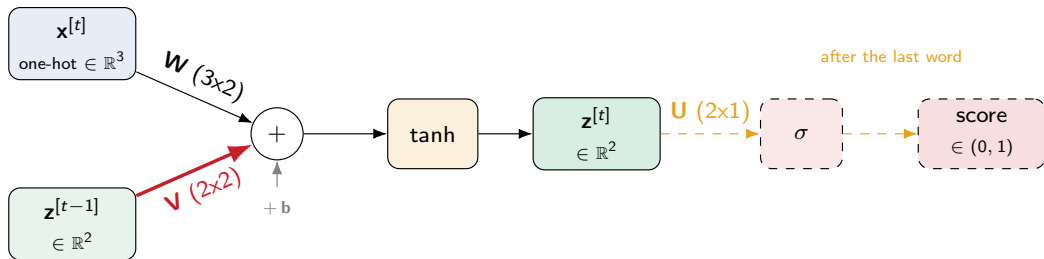
Anatomy of one RNN step

Same recurrence, now with concrete numbers to hold onto: a toy vocabulary of 3 words, a 2-d hidden state. Every arrow below carries its shape, so you always know exactly what multiplies what. **Predict:** the one-hot vector $\mathbf{x}^{[t]}$ meets $\mathbf{W}$ - what does that product actually pick out?

Anatomy of one RNN step

Same recurrence, now with concrete numbers to hold onto: a toy vocabulary of 3 words, a 2-d hidden state. Every arrow below carries its shape, so you always know exactly what multiplies what. **Predict:** the one-hot vector $\mathbf{x}^{[t]}$ meets $\mathbf{W}$ - what does that product actually pick out?

It selects ONE column of $\mathbf{W}^\top$ - equivalently, one row of $\mathbf{W}$. That row/column IS this word's own vector in $\mathbb{R}^2$ - the attention chapter calls it an **embedding**.

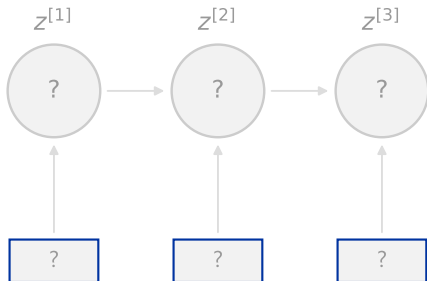


A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. (**Click to advance.**)

Toy net ready: 3-word vocabulary, 2-d state, $z^{[0]} = [0, 0]$

step 1 of 7



tanh applied at every state

W (rows = words)

Ես $[+0.50, -0.50]$

այս $[+1.00, +0.00]$

ամենին $[+0.00, +1.00]$

$V = [[+0.5, -0.5], [+1.0, +0.0]]$

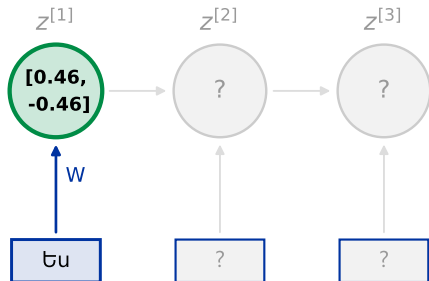
$b = [0, 0]$

A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. (**Click to advance.**)

$x[1] = \text{one-hot}(\text{Ես}) \rightarrow W^T x = \text{row of } W = [0.50, -0.50]$
 $+ V^T z[0] = [0.00, 0.00]$ (state starts at zero) $+ b = [0, 0]$
 $\text{pre} = [0.50, -0.50] \rightarrow \tanh \rightarrow z[1] = [0.46, -0.46]$

step 2 of 7



tanh applied at every state

W (rows = words)

Ես **[+0.50, -0.50]**

այս [+1.00, +0.00]

ամենին [+0.00, +1.00]

$V = [[+0.5, -0.5], [+1.0, +0.0]]$

$b = [0, 0]$

A complete forward pass, in real numbers

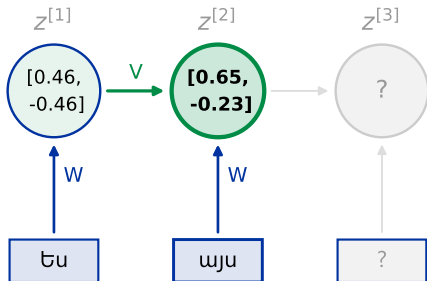
The chapter's Armenian line, first three words, through exactly the net above. (**Click to advance.**)

$x[2] = \text{one-hot}(\omega_{ju}) \rightarrow W^T x = \text{row of } W = [1.00, 0.00]$

step 3 of 7

$+ V^T z[1] = [-0.23, -0.23] + b = [0, 0]$

$\text{pre} = [0.77, -0.23] \rightarrow \tanh \rightarrow z[2] = [0.65, -0.23]$



tanh applied at every state

W (rows = words)

Ես $[+0.50, -0.50]$

այս $[+1.00, +0.00]$

ամենին $[+0.00, +1.00]$

$V = [[+0.5, -0.5], [+1.0, +0.0]]$

$b = [0, 0]$

A complete forward pass, in real numbers

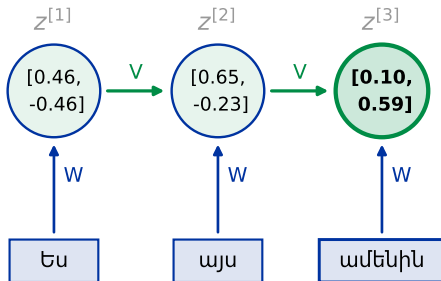
The chapter's Armenian line, first three words, through exactly the net above. (**Click to advance.**)

$$x[3] = \text{one-hot}(\text{ամենիկ}) \rightarrow W^T x = \text{row of } W = [0.00, 1.00]$$

step 4 of 7

$$+ V^T z[2] = [0.10, -0.32] + b = [0, 0]$$

$$\text{pre} = [0.10, 0.68] \rightarrow \tanh \rightarrow z[3] = [0.10, 0.59]$$



tanh applied at every state

W (rows = words)

Ես $[+0.50, -0.50]$

այս $[+1.00, +0.00]$

ամենիկ $[+0.00, +1.00]$

$V = [[+0.5, -0.5], [+1.0, +0.0]]$

$b = [0, 0]$

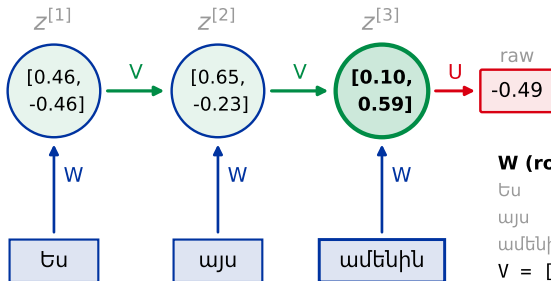
A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. (**Click to advance.**)

After the last word: readout $U^T z^{[3]} + c$

step 5 of 7

(it happens once, not at every step)



tanh applied at every state

W (rows = words)

Ես $[+0.50, -0.50]$

այս $[+1.00, +0.00]$

ամենին $[+0.00, +1.00]$

$V = [[+0.5, -0.5], [+1.0, +0.0]]$

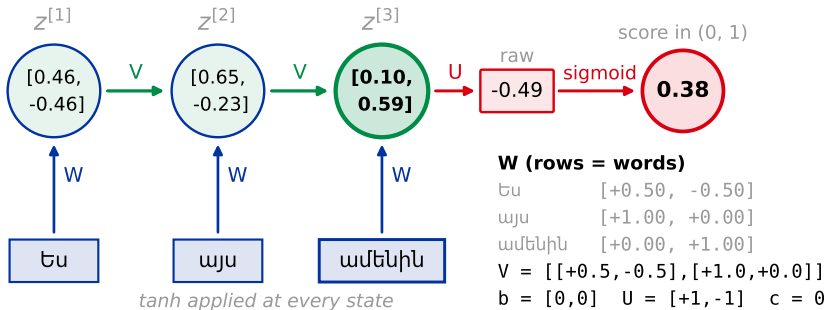
$b = [0, 0]$ $U = [+1, -1]$ $c = 0$

A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. (**Click to advance.**)

$\text{sigmoid}(-0.49) = 0.38$: one pass, one score

step 6 of 7



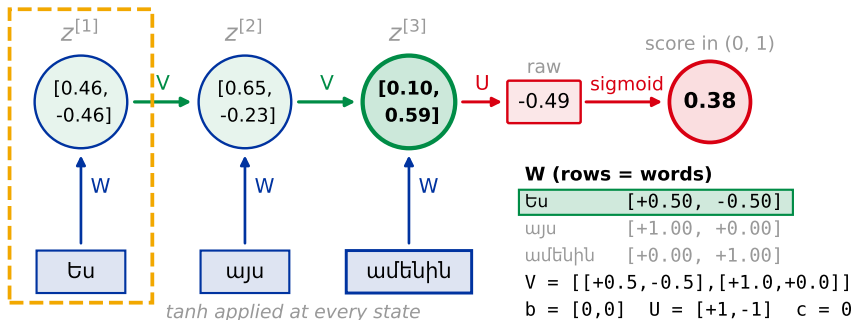
A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. (**Click to advance.**)

one-hot x W selects ONE row of W : that row IS Ես 's own vector

step 7 of 7

The attention chapter gives this row a name: an embedding.



Forward pass, summarized

All values rounded to 2 decimals; $\mathbf{z}^{[0]} = [0, 0]$:

$$\mathbf{z}^{[1]} = \tanh(\mathbf{W}^\top \mathbf{x}_1 + \mathbf{b}) = [0.46, -0.46]$$

$$\mathbf{z}^{[2]} = \tanh(\mathbf{V}^\top \mathbf{z}^{[1]} + \mathbf{W}^\top \mathbf{x}_2 + \mathbf{b}) = [0.65, -0.23]$$

$$\mathbf{z}^{[3]} = \tanh(\mathbf{V}^\top \mathbf{z}^{[2]} + \mathbf{W}^\top \mathbf{x}_3 + \mathbf{b}) = [0.10, 0.59]$$

$$\text{score} = \sigma(\mathbf{U}^\top \mathbf{z}^{[3]}) = \sigma(-0.49) = 0.38$$

Three words, three applications of the **same** $\mathbf{W}$ and $\mathbf{V}$, one final readout through $\mathbf{U}$. Nothing hidden - every number above came straight out of the animation.

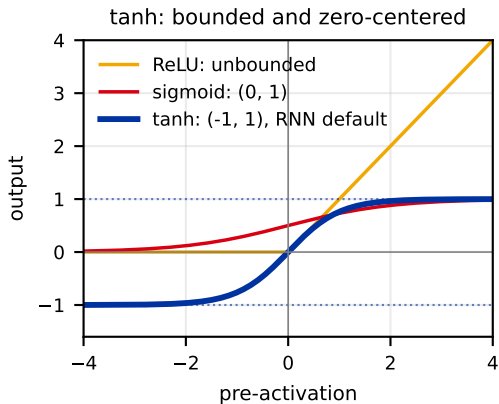
In practice $d_h = 64$ -512, not 2 - but the recipe is identical. And it is **fixed**: 3 words or 3,000, the whole past must fit into those d_h numbers - a lossy summary L21 will come back to.

Picking the activation: why tanh

Three candidates for the state's activation - only one is the RNN default:

- ▶ **tanh** (default): bounded in $(-1, 1)$ - the state literally **cannot blow up** across steps; zero-centered, so both signs flow.
- ▶ **sigmoid**: also bounded, but $|\sigma'| \leq 1/4$ everywhere - one more shrinking factor on top of **V** every step (the "full derivation, part 2" frame ahead uses exactly this bound).
- ▶ **ReLU**: unbounded above - feed that back through **V** every step and the state can genuinely **explode**, not just saturate.

Dead ReLU, one line: a unit whose pre-activation is always ≤ 0 outputs zero forever, and its gradient too - it never updates again.

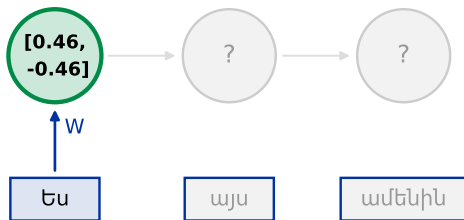


Unrolling

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Forward: ես -> այս -> ամենին

$$z[1] = \tanh(W^T x) = [0.46, -0.46]$$



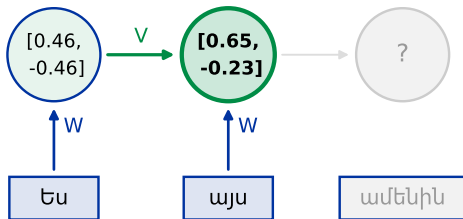
step 1 of 6

Unrolling

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Forward: ես -> այս -> ամենին

$$z[2] = \tanh(V^T z[1] + W^T x + b) = [0.65, -0.23]$$



step 2 of 6

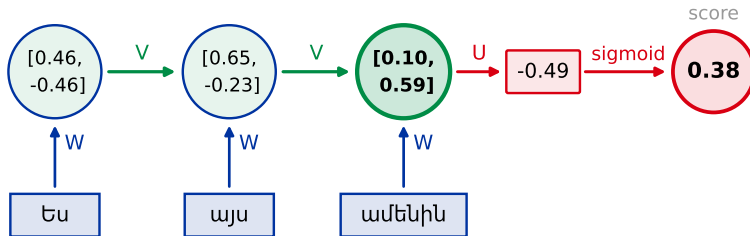
Unrolling

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Forward: ես -> այս -> ամենին

$$z[3] = \tanh(V^T z[2] + W^T x + b) = [0.10, 0.59]$$

forward score = 0.38



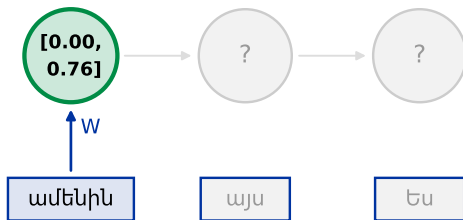
step 3 of 6

Unrolling

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Reversed: ամենին -> այս -> ես

$$z[1]' = \tanh(W^T x) = [0.00, 0.76]$$



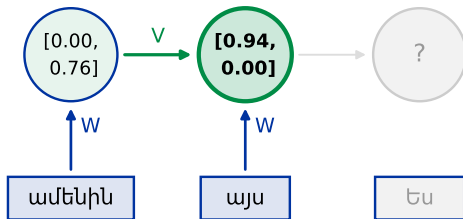
step 4 of 6

Unrolling

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Reversed: ամենին -> այս -> ես

$$z[2]' = \tanh(V^T z[1]' + W^T x + b) = [0.94, 0.00]$$



step 5 of 6

Unrolling

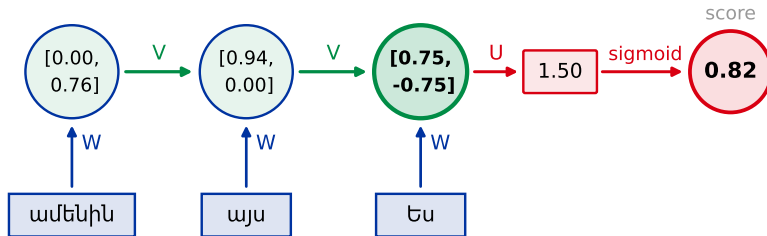
The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Reversed: ամենին -> այս -> էս

$$z[3]' = \tanh(V^T z[2]' + W^T x + b) = [0.75, -0.75]$$

reversed score = 0.82

Same 3 words, different order: 0.38 vs 0.82



step 6 of 6

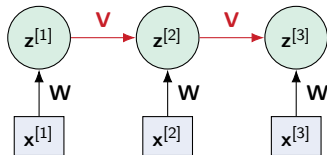
Parameter sharing

The *same* $\mathbf{W}$ and $\mathbf{V}$ at **every** step.
Parameter count is independent of sequence length: with $d_x = 1$, $d_h = 16$,

$$\underbrace{16}_W + \underbrace{256}_V + \underbrace{16}_b + \underbrace{16}_U + \underbrace{1}_c = 305$$

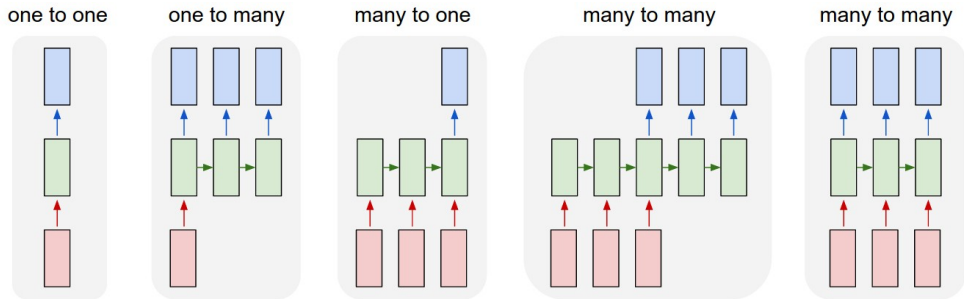
parameters - whether the review is 3 words or 40.

Convolution shares across **space** ([48]); recurrence shares across **time**. Same trick, new axis - and it kills hack (3): a pattern learned at position 2 works at position 17 for free.



Every arrow of the same color is the *same* matrix.

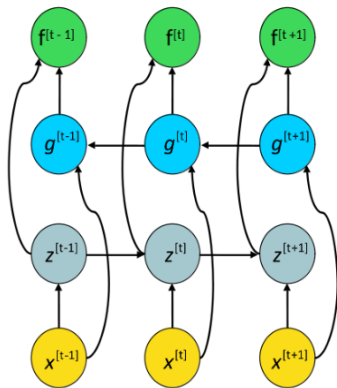
The taxonomy



- ▶ **seq-to-one:** sentiment (our reviews), document classification.
- ▶ **one-to-seq:** image captioning.
- ▶ **seq-to-seq:** translation, language modeling - the vocabulary L21 needs.

Variants exist - bidirectional (next slide), stacked (deeper). The diagram's "many-to-X" naming = our "seq-to-X": same taxonomy. Diagram: Karpathy (2015), "The Unreasonable Effectiveness of Recurrent Neural Networks" - L21 returns to that essay.

Reading both ways: bidirectional RNNs



The **forward** states $z^{[t]}$ run left→right; the **backward** states $g^{[t]}$ run right→left; each output $f^{[t]}$ reads *both*.

A plain RNN only sees the **past**. But the right reading of a word often depends on what comes *after*:

- “**Teddy** Roosevelt” vs “**Teddy** bears” - is “Teddy” a name? You only know from the *next* word.

A **bidirectional** RNN (Schuster & Paliwal, 1997) runs one RNN each way and concatenates their states, so **every position sees the whole sequence**.

Great for **encoding** a sequence you already have (tagging, sentiment, the encoder half of translation). The catch: it needs the *whole input up front*, so it cannot **generate** or stream token by token - the setting L21 cares about.

Training - and the fatal flaw

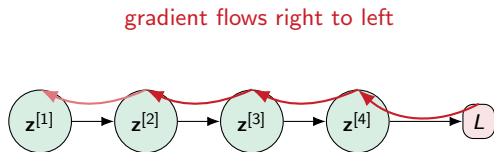
Unroll it, and it is just a deep network.

Backprop through time

The unrolled RNN is a deep feed-forward net with **tied weights**. Autograd needs nothing new - [42]'s `loss.backward()` (one backward sweep, any graph), kept again. A tied weight collects the **sum** of its gradients from every step - the same rule as a shared conv kernel ([48]).

For seq-to-one sentiment, the loss sits only at the **final** step - so its gradient must travel all the way back through every recurrent connection.

Formalized as "backpropagation through time" by Werbos (1990) - four years after backprop itself (Rumelhart, Hinton & Williams, 1986).



Predict-first: the fading number

Strip the net down to 1-d to see the decay cleanly: a scalar state

$h^{[t]} = Vh^{[t-1]} + Wx^{[t]}$, no vectors,
no activation. On a 30-word review,
how much does token 1 influence
the state at token 30?

$$\frac{dh_{30}}{dh_1} = V^{29} = 0.8^{29}$$

Guess the value.

Predict-first: the fading number

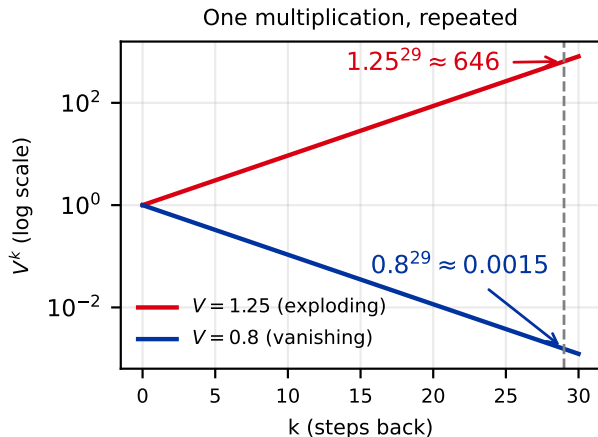
Strip the net down to 1-d to see the decay cleanly: a scalar state
 $h^{[t]} = Vh^{[t-1]} + Wx^{[t]}$, no vectors,
no activation. On a 30-word review,
how much does token 1 influence
the state at token 30?

$$\frac{dh_{30}}{dh_1} = V^{29} = 0.8^{29}$$

Guess the value.

$0.8^{29} \approx \mathbf{0.0015}$. And if $V = 1.25$?
 $1.25^{29} \approx \mathbf{646}$. One multiplication, re-
peated, either starves or explodes.

[29]'s smoothing had the same 0.8. There,
fading is the point; in language it is the
disease.



The full derivation, part 1

For vectors, the state update is $\mathbf{z}^{[t]} = \sigma(\mathbf{V}^\top \mathbf{z}^{[t-1]} + \mathbf{W}^\top \mathbf{x}^{[t]} + \mathbf{b})$, so by the chain rule:

$$\frac{d\mathbf{z}^{[t]}}{d\mathbf{z}^{[t-1]}} = \text{diag}(\sigma'(\cdot)) \mathbf{V}^\top = \mathbf{D}^{[t-1]} \mathbf{V}^\top$$

Chain this back to step 1:

$$\frac{dL}{d\mathbf{z}^{[1]}} = \frac{dL}{d\mathbf{z}^{[t]}} \mathbf{D}^{[t-1]} \mathbf{D}^{[t-2]} \dots \mathbf{D}^{[1]} (\mathbf{V}^\top)^{t-1}$$

Every path back to an early time-step runs through powers of the **same** matrix $\mathbf{V}^\top$. For any $i < t$: the term $(\mathbf{V}^\top)^{t-i}$ appears.

Why eigenvalues? Along an eigenvector $\mathbf{v}$, the matrix only scales: $\mathbf{V}^\top \mathbf{v} = \lambda \mathbf{v}$, so $(\mathbf{V}^\top)^k \mathbf{v} = \lambda^k \mathbf{v}$. In that direction the matrix power *is* the scalar story, 0.8^{29} again - and after many steps the direction with the largest $|\lambda|$ dominates everything else.

The full derivation, part 2 - eigenvalues

The **largest eigenvalue** $|\lambda_{\max}|$ of $\mathbf{V}$ decides the fate of $(\mathbf{V}^\top)^{t-i}$:

- ▶ $|\lambda_{\max}| < 1 \Rightarrow$ **vanishing** - and the $\mathbf{D}$ factors (≤ 1 for tanh) can only shrink it further.
- ▶ $|\lambda_{\max}| > 1 \Rightarrow$ it **can explode** - unless saturated $\mathbf{D}$ factors happen to cancel the growth.

(Pascanu, Mikolov & Bengio, 2013, make it precise: with tanh, $|\lambda_{\max}| < 1$ is *sufficient* for vanishing, while $|\lambda_{\max}| > 1$ is only *necessary* for exploding.)

Worse than a deep MLP. An MLP has a *different* matrix per layer - errors can partially cancel. An RNN multiplies by the **same** $\mathbf{V}$ at every step, so the effect compounds cleanly. The sigmoid's $\mathbf{D}$ factors only make it worse ($|\sigma'| \leq 1/4$).

Callback: [42] already met vanishing gradients in deep MLPs (the δ recursion), and [45] fought them with ReLU and He init. Same disease, worse here - it is the *same* matrix every step.

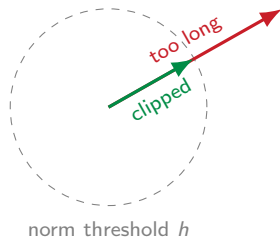
History: this diagnosis is Hochreiter's 1991 diploma thesis (written in German, barely read at the time) and, independently, Bengio, Simard & Frasconi (1994). Remember the name Hochreiter - he returns next lecture with the cure.

Exploding is the easy half

[44] already gave us the fix for cliffs - **gradient clipping**: cap the gradient's norm at a threshold h :

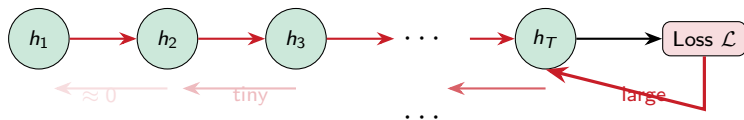
$$\text{if } \|\nabla\| > h: \quad \nabla \leftarrow \frac{h}{\|\nabla\|} \nabla$$

One formula (Pascanu, Mikolov & Bengio, 2013), solved problem. Exploding gradients are the *easy* half of this story.



What vanishing means in practice

Backpropagation through time



$$\frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial h_T} \cdot \prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \quad \text{product}$$

of many values $< 1 \Rightarrow$ gradient vanishes

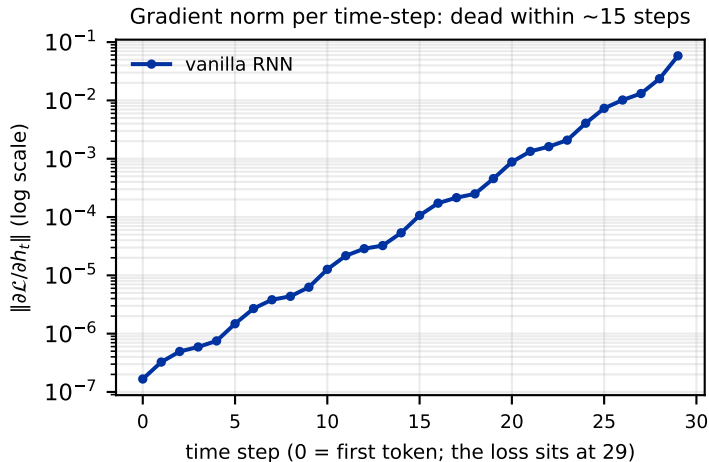
"The cat that I saw yesterday in the garden near the old oak tree **was** cute"
- the gradient from "was" can't reach "cat": the RNN *forgets* early tokens

Our version: the 42-word review from the shoehorn-hacks frame - its verdict, *disappointing*, sits in the last clause. By the time the RNN reaches it, the state has forgotten most of what came before.

Empirical proof

Gradient norm per time-step, measured directly on a tiny vanilla RNN (hidden size 16). The task: read 30 random numbers, then output the **first** one - so the loss at the end needs the gradient to reach step 1.

Not asserted - **measured**. Note the log scale: the gradient reaching step 1 is over **300,000** $\times$ smaller than at the end - and it has already lost 1000 $\times$ within the last 15 steps.



Recap

- ▶ Text refuses to fit a fixed input: length varies, order matters - every shoehorn hack loses one or the other, and a fixed window is blind beyond k .
- ▶ State = a running summary; **one new weight matrix V** turns a dense layer into an RNN.
- ▶ Unrolled = a deep network with **tied weights** - autograd handles it unchanged.
- ▶ Same-matrix powers $\Rightarrow$ gradients **vanish** (derived, not asserted) or explode.
- ▶ Clipping fixes exploding. **Nothing yet fixes vanishing** - so a vanilla RNN's "unlimited memory" is, in practice, a few dozen steps: no better than a wide fixed window.

Next: 1997's answer - Hochreiter & Schmidhuber's **Long Short-Term Memory (LSTM)**, a memory built from **addition**, not multiplication. It later runs Google Translate and Siri. And it still will not be enough (L21).